Space Mission Control

SMC

Architecture API & Gestion des données

Fonctionnement détaillé du backend Spring Boot

Stack : Spring Boot 3.x · Java 25 · JWT · JPA/Hibernate · MySQL

Avril 2026

Table des matières

1 Vue d'ensemble de l'architecture **2**
1.1 Schéma en couches . . . 2
1.2 Structure des packages Java . . . 2

2 Authentification JWT **2**
2.1 Connexion — POST /api/auth . . . 3
2.2 Requêtes authentifiées — Flux du filtre JWT . . . 3
2.3 Structure du token JWT . . . 3

3 Modèle de données **4**
3.1 Entités JPA et relations . . . 4
3.2 Données de référence . . . 4
3.3 Cycle de vie d'une mission . . . 5

4 Endpoints REST **5**
4.1 Authentification . . . 5
4.2 Utilisateurs . . . 5
4.3 Missions . . . 5
4.4 Spacecraft, Données de référence & Trajectoires . . . 6
4.5 Actions sur un Spacecraft . . . 6

5 Pattern DTO **8**
5.1 Exemple — Création de mission . . . 8

6 Sécurité & Contrôle d'accès par rôle (RBAC) **8**
6.1 Matrice des autorisations . . . 8
6.2 Mécanismes Spring Security . . . 9

7 Calcul de trajectoire orbitale **9**
7.1 Flux de données . . . 9
7.2 Formules de mécanique orbitale newtonienne . . . 10

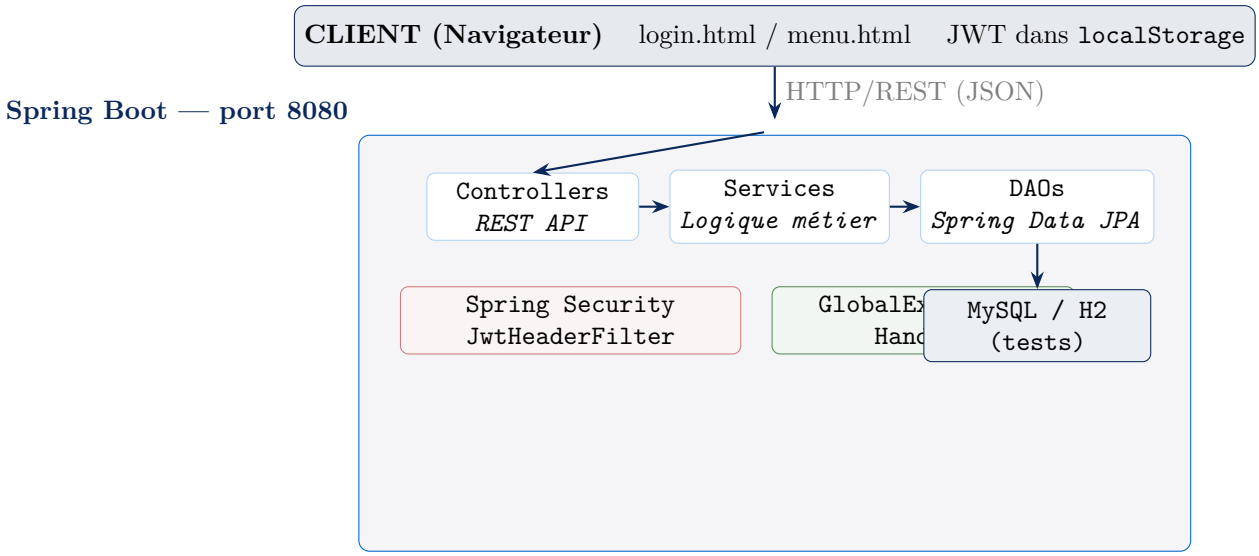
8 Gestion des exceptions **10**

9 Cycle de vie d'une requête **10**

1 Vue d'ensemble de l'architecture

L'application est organisée en **couches strictement séparées**, chacune ayant une responsabilité unique. Cette séparation garantit la testabilité, la maintenabilité et la sécurité.

1.1 Schéma en couches



1.2 Structure des packages Java

```
space/
        Application.java          <- @SpringBootApplication (point d'entree)
        config/
             SecurityConfig.java    <- Regles CORS, CSRF, JWT, RBAC
             JwtUtils.java          <- Generation / validation des tokens
             JwtHeaderFilter.java   <- Filtre HTTP : lit le Bearer token
             JpaUserDetailsService.java<- Bridge Spring Security <-> BDD
        rest/                       <- 7 controleurs REST
        service/
             TrajectoryService.java <- Orchestration calcul orbital
        simulation/orbital/
             OrbitalCalculator.java <- Physique newtonienne (POJO pur)
             TrajectoryResult.java  <- DTO interne du resultat
        model/                      <- 8 entites JPA
        dao/                        <- 6 interfaces Spring Data JPA
        dto/request/                <- 8 classes de requetes entrantes
        dto/response/               <- 7 classes de reponses sortantes
        exception/                  <- 5 fichiers (4 exceptions + handler)
```

2 Authentification JWT

2.1 Connexion — POST /api/auth

1. Requête `POST /api/auth`
Corps JSON : `{ "mail": "admin@smc.fr", "password": "admin123" }`

↓

2. AuthenticationManager.authenticate()
→ `JpaUserDetailsService.loadUserByUsername(mail)`
→ Charge l'utilisateur depuis la BDD
→ Comparaison `BCrypt(password, hash_en_base)`

↓

3. JwtUtils.generate(authentication)
→ Token signé HS512 · sujet = mail · expiration = now + 8h

↓

4. Réponse `200 OK`
Corps JSON : `{ "token": "eyJhbGciOiJIUzUxMiJ9..." }`
Client stocke dans `localStorage["jwt"]`

2.2 Requêtes authentifiées — Flux du filtre JWT

Pour toute requête sur `/api/**` (sauf `/api/auth`), le filtre `JwtHeaderFilter` s'exécute **avant** les contrôleurs :

1. Lecture de l'en-tête
`Authorization: Bearer eyJhbGci...`

↓

2. JwtUtils.validate(token)
Vérifie la signature HMAC-512 et la date d'expiration
→ Extrait le sujet (mail de l'utilisateur)

↓

3. Chargement depuis la BDD
`IDAOUtilisateur.findByMail(mail)`
→ Récupère le rôle *actuel* (ADMIN / OPERATEUR)

↓

4. Peuplement du SecurityContext
`UsernamePasswordAuthenticationToken(mail, null, [ROLE_ADMIN])`
→ Spring Security peut vérifier les autorisations

2.3 Structure du token JWT

Un token JWT est composé de trois parties séparées par des points (`.`) :

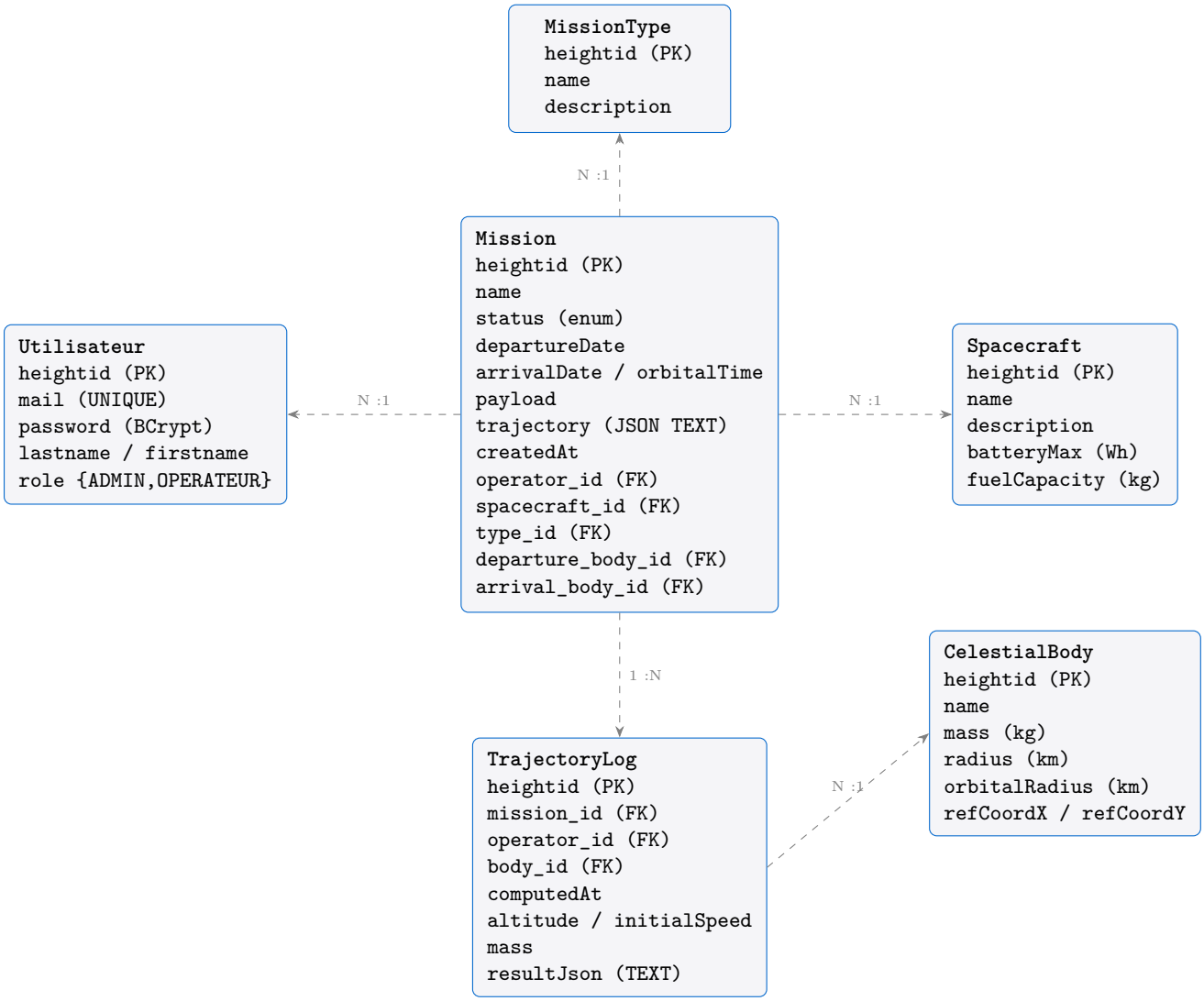
Anatomie d'un JWT

Partie	Contenu	Rôle
Header	`{"alg":"HS512","typ":"JWT"}`	Algorithme de signature
Payload	`{"sub":"admin@smc.fr","iat":...,"exp":...}`	Données (mail, dates)
Signature	HMAC-SHA512(header+payload, clé 64 octets)	Intégrité / authenticité

Durée de validité : 8 heures (28 800 000 ms) · **Algorithme :** HS512

3 Modèle de données

3.1 Entités JPA et relations



3.2 Données de référence

Ces données sont pré-chargées par `init_smc_db.sql` et ne sont jamais modifiées via l'API :

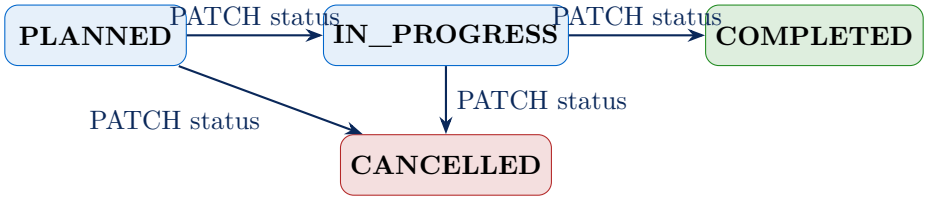
Corps célestes (CelestialBody)

Soleil · Mercure · Vénus · **Terre** · **Mars** · Jupiter · Saturne · Uranus · Neptune

Types de mission (MissionType)

Orbitale LEO · Orbitale GEO · Lunaire · Interplanétaire

3.3 Cycle de vie d'une mission



La transition de statut est réservée aux **ADMIN** (`@PreAuthorize("hasRole('ADMIN')")`).

4 Endpoints REST

4.1 Authentification

Méthode	URL	Description	Accès
`POST`	`/api/auth`	Connexion retourne JWT	Public

4.2 Utilisateurs

Méthode	URL	Description	Accès
`GET`	`/api/utilisateur`	Liste tous les utilisateurs	ADMIN
`GET`	`/api/utilisateur/me`	Profil de l'utilisateur connecté	Tous
`PUT`	`/api/utilisateur/me`	Modifier son mail / mot de passe	Tous
`POST`	`/api/utilisateur`	Créer un utilisateur	ADMIN
`DELETE`	`/api/utilisateur/{id}`	Supprimer un utilisateur	ADMIN

4.3 Missions

Méthode	URL	Description	Accès
`GET`	`/api/mission`	Missions actives (PLANNED + IN_PROGRESS)	Tous*
`GET`	`/api/mission/history`	Missions terminées	Tous*
`GET`	`/api/mission/{id}`	Détail d'une mission	Tous*
`POST`	`/api/mission`	Créer (statut initial : PLANNED)	Tous
`PUT`	`/api/mission/{id}`	Modifier une mission	Tous*
`PATCH`	`/api/mission/{id}/status`	Changer le statut	ADMIN
`DELETE`	`/api/mission/{id}`	Supprimer	ADMIN

*ADMIN voit toutes les missions ; OPERATEUR voit uniquement les siennes.

4.4 Spacecraft, Données de référence & Trajectoires

Méthode	URL	Description	Accès
GET	/api/spacecraft	Liste (avec disponibilité)	Tous
GET	/api/spacecraft/{id}	Détail	Tous
POST	/api/spacecraft	Ajouter	ADMIN
PUT	/api/spacecraft/{id}	Modifier	ADMIN
DELETE	/api/spacecraft/{id}	Supprimer	ADMIN
GET	/api/mission-type	Types de mission	Tous
GET	/api/celestial-body	Corps célestes	Tous
GET	/api/celestial-body/{id}	Détail corps céleste	Tous
POST	/api/trajectory/{missionId}	Calculer une trajectoire	Tous
GET	/api/trajectory/{missionId}	Historique des calculs	Tous

4.5 Actions sur un Spacecraft

Méthode	URL	Description	Accès
POST	/api/action/{spacecraftId}	Déclencher une action sur un spacecraft	Tous

Le type d'action est transmis dans le corps de la requête. L'API vérifie d'abord que le spacecraft est **capable** d'exécuter cette action avant de déclencher le code technique correspondant et de recalculer la trajectoire orbitale.

Valeurs possibles de `actionType` :

Enumération TYPE_ACTION

Valeur	Effet sur la simulation	Spacecraft
CHANGER_PROPULSION	Modifie Δv : recalcul trajectoire	Satellite
CHANGER_DIRECTION	Modifie Δv : recalcul trajectoire	Satellite
OUVRIR_PANNEAU	recharge batterie	Satellite
TRANSMISSION_DONNES	conso. électrique	Satellite
SCAN_SURFACE	conso. électrique	Satellite
MODE_ECO	conso. électrique réduite	Satellite
CHANGER_PROPULSION	Modifie Δv : recalcul trajectoire	Pod Habité
CHANGER_DIRECTION	Modifie Δv : recalcul trajectoire	Pod Habité
OUVRIR_PANNEAU	recharge batterie	Pod Habité
COLLECTE_DONNES	conso. électrique	Pod Habité
EVA	X	Pod Habité
GESTION_O2	impact batterie	Pod Habité
DEPLACEMENT	conso. électrique	Rover
COLLECTE_DONNES	conso. électrique	Rover
TRANSMISSION_DONNES	conso. électrique	Rover
MODE_ECO	conso. électrique réduite	Rover
PHOTO	impact batterie	Rover
DEPLACEMENT	conso. électrique	Utilitaire
MAINTENANCE	?	Utilitaire
PROD_ELEC	recharge batterie	Utilitaire

1. Requête `POST /api/action/{spacecraftId}`
Corps JSON : `{ "actionType": "CHANGER_PROPULSION" }`

↓

2. Vérification des capacités (ActionRegistry)
→ `canExecute(TYPE_ACTION, spacecraft)`
→ Vérifie dans `capacities : Map<TYPE_ACTION, Spacecraft>`
→ Si spacecraft non capable lance `ActionNotSupportedException` (403)

↓

3. Exécution de l'action (ActionRegistry MoteurPhysique)
→ `execute(TYPE_ACTION, spacecraft)`
→ Passe l'instance `Runnable` (*Action) au `MoteurPhysique`
→ `MoteurPhysique.triggerAction(action)` appelle `run()`
Exemple : `ChangerPropulsionAction.run()` modifie le delta_v

↓

4. Mise à jour des consommables (Spacecraft)
→ `spacecraft.updateConsummable(TYPE_ACTION)`
→ Déduit la consommation carburant / batterie selon le type d'action

↓

5. Recalcul orbital (MoteurPhysique)
→ `eulerOrbitInit()` — initialisation de l'orbite
→ `perturbateOrbit(currentOrbit)` — intégration numérique (Kepler + méthode de Cowell, Euler/RK4)
→ Retourne l'orbite mise à jour

↓

6. Réponse `200 OK`
Corps JSON : trajectoire orbitale recalculée

Exemple de requête et réponses :

Requête `POST /api/action/3` :

```
{
  "actionType": "CHANGER_PROPULSION"
}
```

Réponse `200 OK` :

```
{
  "spacecraftId": 3,
  "actionType": "CHANGER_PROPULSION",
  "updatedOrbit": {
    "trajectoire": [...],
    "vectorSpeed": [...],
    "vectorAccel": [...]
  }
}
```

Réponse `403 Forbidden` :

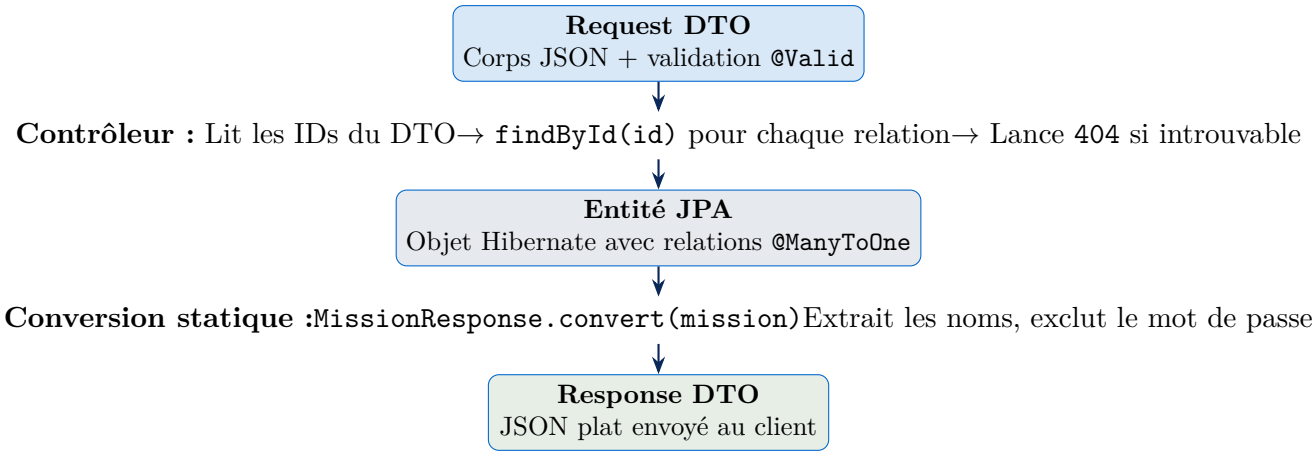
```
{
  "error": "Action CHANGER_PROPULSION
  non supportee par le spacecraft id:3"
}
```

Réponse `404 Not Found` :

```
{
  "error": "Spacecraft introuvable
      id : 3"
}
```

5 Pattern DTO

Le pattern DTO (*Data Transfer Object*) découple la structure de la base de données du contrat de l'API.



5.1 Exemple — Création de mission

Requête `POST /api/mission` :

```
{
  "name": "Mars Alpha",
  "spacecraftId": 2,
  "typeId": 4,
  "departureBodyId": 1,
  "arrivalBodyId": 4,
  "departureDate": "2026-06-01T08:00",
  "payload": 1500.0
}
```

Réponse `201 Created` :

```
{
  "id": 42,
  "name": "Mars Alpha",
  "status": "PLANNED",
  "operatorMail": "admin@smc.fr",
  "spacecraftName": "Ariane 6",
  "typeName": "Interplanetaire",
  "departureBodyName": "Soleil",
  "arrivalBodyName": "Mars",
  "createdAt": "2026-04-11T14:30:00"
}
```

6 Sécurité & Contrôle d'accès par rôle (RBAC)

6.1 Matrice des autorisations

Action	ADMIN	OPERATEUR
Voir *toutes* les missions	✓	×
Voir ses propres missions	✓	✓
Créer une mission	✓	✓
Modifier une mission	✓	Les siennes
Changer le statut d'une mission	✓	×
Supprimer une mission	✓	×
Voir tous les spacecraft	✓	✓
Créer / modifier / supprimer spacecraft	✓	×
Gérer les utilisateurs	✓	×
Calculer une trajectoire orbitale	✓	✓
Déclencher une action sur un spacecraft	✓	✓

6.2 Mécanismes Spring Security

La sécurité est appliquée à **deux niveaux** :

1. **Niveau URL** (dans `SecurityConfig`) : toute route `/api/**` requiert un JWT valide.
2. **Niveau méthode** (annotation `@PreAuthorize`) : restreint l'accès à des rôles précis.

```
// Exemple dans SpacecraftController
@DeleteMapping("/{id}")
@PreAuthorize("hasRole('ADMIN')")      // Spring Security verifie le rle
public ResponseEntity<Void> delete(@PathVariable int id) {
    // ...
}

// Exemple de filtrage dynamique dans MissionController
String role = SecurityContextHolder.getContext()
    .getAuthentication().getAuthorities().toString();

List<Mission> missions = role.contains("ADMIN")
    ? dao.findAll()                 // ADMIN : toutes les missions
    : dao.findByOperator(utilisateur); // OPERATEUR : seulement les siennes
```

7 Calcul de trajectoire orbitale

7.1 Flux de données

1. Requête `POST /api/trajectory/{missionId}`
Corps : `{ altitude: 400, initialSpeed: 7669, spacecraftMass: 20000, celestialBodyId: 3 }`

↓

2. Chargement des entités (TrajectoryService)
→ Charge `Mission` par `missionId` (404 si absent)
→ Charge `CelestialBody` (Terre : M = $5{,}9710^{24}$ kg, R = 6 371 km)
→ Charge `Utilisateur` courant (pour l'audit)

↓

3. OrbitalCalculator.compute(altitude, masse, rayon)
$r = (6371 + 400) \times 1000 = 6\,771\,000$ m
$v = \sqrt{GM/r} \approx 7\,669$ m/s $T = 2\pi\sqrt{r^3/GM} \approx 5\,559$ s
360 points `[x,y]` = `[r·cos(), r·sin()]` pour [0, 360]

↓

4. Persistance (TrajectoryService)
→ `ObjectMapper.writeValueAsString(result)` JSON
→ Sauvegarde `TrajectoryLog` (audit complet)
→ Met à jour `Mission.trajectory` (dernier résultat)

↓

5. Réponse `200 OK` `TrajectoryLogResponse`
Contient : vitesse, période, rayon, 360 points de trajectoire

7.2 Formules de mécanique orbitale newtonienne

Formules clés — OrbitalCalculator.java

$$r = (R_{\text{planète}} + h) \times 1000 \quad [\text{m}] \tag{1}$$

$$v_{\text{orb}} = \sqrt{\frac{G \cdot M}{r}} \quad [\text{m/s}] \tag{2}$$

$$T = 2\pi\sqrt{\frac{r^3}{G \cdot M}} \quad [\text{s}] \tag{3}$$

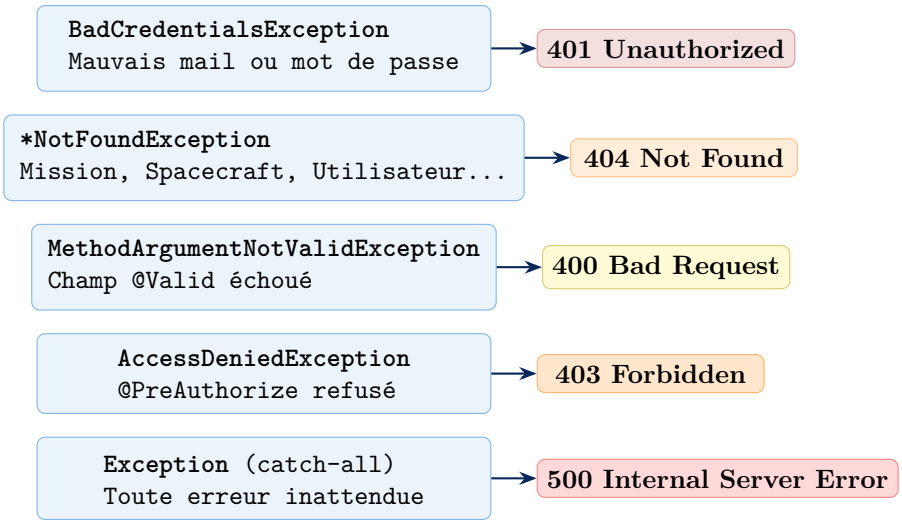
$$\text{Point}_\theta = (r\cos\theta,\ r\sin\theta) \quad \theta \in \{0, 1, \ldots, 359\} \tag{4}$$

Avec $G = 6{,}674 \times 10^{-11}\ \text{m}^3 \cdot \text{kg}^{-1} \cdot \text{s}^{-2}$ et M = masse du corps céleste (kg)

Exemple ISS (Terre, $h = 400$ km) : $r \approx 6\,771\,000$ m $v \approx 7\,669$ m/s $T \approx 5\,559$ s (92 min)

8 Gestion des exceptions

Toutes les exceptions non interceptées dans les contrôleurs remontent à `GlobalExceptionHandler` (`@RestControllerAdvice`), qui les convertit en réponses JSON standardisées.



Format de réponse JSON :

```
// Erreur simple (401, 403, 404, 500)
{ "error": "Mission introuvable  id :42" }

// Erreur de validation (400)  un message par champ invalide
{ "name": "Le nom est obligatoire", "mail": "Format email invalide" }
```

9 Cycle de vie d'une requête

Exemple complet : `GET /api/mission` effectué par un utilisateur **OPERATEUR**.

Browser envoie `GET /api/mission`
`Authorization: Bearer eyJhbGci...`

↓

[1] Couche CORS / Preflight
OPTIONS préalable autorisé sans authentification (règle `requestMatchers(OPTIONS, /**).permitAll()`)

↓

[2] JwtHeaderFilter
→ Lit `Authorization: Bearer ...`
→ `JwtUtils.validate()` extrait `"op@smc.fr"`
→ `IDAOUtilisateur.findByMail()`
`Utilisateur{role=OPERATEUR}`
→ Peuple `SecurityContextHolder`

↓

[3] Spring Security — vérification des règles
→ `/api/**  authenticated()` OK (token valide)
→ Pas de `@PreAuthorize` sur `GET /api/mission` OK

↓

[4] MissionController.getAll()
→ Lit le rôle depuis `SecurityContextHolder`
→ Rôle = `OPERATEUR`
`IDAOMission.findByOperator(utilisateur)`
→ Retourne uniquement les missions de cet opérateur

↓

[5] Conversion DTO
→ `MissionResponse.convert(mission)` pour chaque mission
→ Noms extraits, mot de passe exclu, JSON plat

↓

[6] Réponse HTTP
`200 OK · Content-Type: application/json`
`[{"id":1,"name":"ISS Repair","status":"PLANNED",...}]`

Annexes

Comptes de test

Mail	Mot de passe	Rôle
`admin@smc.fr`	`admin123`	ADMIN
`op@smc.fr`	`op123`	OPERATEUR

Lancement de l'application

```
# Lancer le serveur Spring Boot
cd ~/Desktop/SPACE/projet
./mvnw spring-boot:run
#  Accs  : http://localhost:8080/login/login.html

# Lancer les tests (41 tests  BUILD SUCCESS)
./mvnw test
```